



ZPI-Desktop-App

Dokumentacja plików — katalog `src/`

103 pliki .ts/.html/.scss

Angular 20 Standalone

Ionic 8

TypeScript

Projekt Inżynierski — ZPI | 2025/2026 | Wersja 1.0

Spis treści

1. Pliki główne src/	10. features/konflikty/
2. app.component + app.routes	11. features/sale/
3. core/auth/	12. features/subjects/
4. core/interceptors/	13. features/teaching-loads/
5. core/services/	14. features/admin/
6. core/validators/	15. features/chat/
7. pages/ (login, change-password)	16. features/audit/
8. features/home/	17. features/profile/
9. features/hermonogram/	

1. Pliki główne src/

main.ts

src/main.ts

Punkt wejścia całej aplikacji. To pierwszy plik uruchamiany przez przeglądarkę. Inicjalizuje Angulara i rejestruje globalne dostawców.

```
bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
});
```

Element	Co robi
<code>bootstrapApplication</code>	Uruchamia aplikację od komponentu <code>AppComponent</code> (bez modułu NgModule)
<code>IonicRouteStrategy</code>	Strategia reużywania tras zoptymalizowana pod Ionic (zachowuje stan stron w historii)
<code>provideIonicAngular()</code>	Rejestruje wszystkie komponenty Ionic w aplikacji
<code>withPreloading(PreloadAllModules)</code>	Po starcie aplikacji w tle preładuje wszystkie lazy-loaded moduły
<code>withInterceptors([authInterceptor])</code>	Rejestruje interceptor HTTP globalnie dla wszystkich żądań

environment.ts

src/environments/environment.ts

Konfiguracja środowiska deweloperskiego. Definiuje zmienne dostępne w całej aplikacji przez import `environment`.

```
const apiHost = typeof window !== 'undefined' ? window.location.hostname : '127.0.0.1';
export const environment = {
  production: false,
  apiUrl: `http://${apiHost}:8000`,
};
```

Kluczowy trick

`window.location.hostname` — adres URL backendu jest automatycznie wykrywany z hosta przeglądarki. Jeśli aplikacja działa na `192.168.1.10:4200`, API będzie pod `http://192.168.1.10:8000`. Działa zarówno lokalnie jak i w sieci lokalnej bez rekonfiguracji.

environment.prod.ts

src/environments/environment.prod.ts

Konfiguracja produkcyjna. Zastępuje `environment.ts` podczas budowania (`ng build --configuration production`). Identyczna logika — `production: true` wyłącza narzędzia deweloperskie Angulara i włącza optymalizacje.

global.scss

src/global.scss

Globalne style CSS aplikowane do wszystkich stron. Zawiera importy zmiennych Ionic, globalne resety i wspólne klasy pomocnicze.

theme/variables.scss

src/theme/variables.scss

Zmienne CSS (custom properties) dla motywu Ionic — kolory, fonty, rozmiary komponentów. Zawiera definicje `--ion-color-primary`, `--ion-color-secondary` itp. dla trybu jasnego i ciemnego (`@media (prefers-color-scheme: dark)`).

index.html

src/index.html

Jedyna strona HTML aplikacji SPA. Zawiera tag `<app-root>` (punkt montowania Angulara) oraz meta tagi. Angular CLI automatycznie wstrzykuje do niej tagi `<script>` wskazujące na chunki JS podczas budowania.

polyfills.ts

src/polyfills.ts

Importuje polyfills — kod uzupełniający brakujące funkcje w starszych przeglądarkach (np. `zone.js` wymagany przez Angulara do wykrywania zmian).

zone-flags.ts

src/zone-flags.ts

Flagi konfiguracyjne dla `zone.js` — pozwalają wyłączyć monkeypatching pewnych API (np. `__Zone_disable_requestAnimationFrame`) w celu poprawy wydajności.

2. `app.component` + `app.routes.ts`

`app.component.ts`

src/app/app.component.ts

Główny komponent aplikacji — root. Renderowany jako pierwszy, zawiera globalne elementy UI widoczne na każdej stronie.

Właściwość / Metoda	Opis
<code>showChatFab</code>	Getter — zwraca <code>true</code> gdy nie jesteśmy na <code>/chat</code> ani <code>/change-password</code> ; kontroluje widoczność pływającego przycisku czatu
<code>isChatPage</code>	Getter — sprawdza czy bieżąca trasa to <code>/chat</code>
<code>navigate(path)</code>	Nawiguje do trasy; jeśli <code>mustChangePassword=true</code> przekierowuje na <code>/change-password</code>
<code>onThemeChange(isDark)</code>	Przełącza motyw jasny/ciemny, zapisuje w preferencjach użytkownika
<code>ngOnInit()</code>	Subskrybuje router events, obserwuje stan autentykacji, uruchamia polling powiadomień co 60s
<code>ngOnDestroy()</code>	Anuluje wszystkie subskrypcje RxJS (zapobiega wyciekowi pamięci)
<code>unreadChatCount</code>	Liczba nieprzeczytanych wiadomości — wyświetlana jako badge na FAB czatu

Zaimportowane ikony (addIcons)

Rejestruje ~30 ikon Ionicons globalnie, tak aby były dostępne w każdym komponencie aplikacji przez nazwę (np. `name="trash-outline"`).

`app.component.html`

src/app/app.component.html

Szablon głównego komponentu. Zawiera trzy elementy:

- `<ion-menu>` — boczne menu nawigacyjne (slide-over); widoczne tylko gdy `auth.isAuthenticated$ | async` = true; elementy menu filtrowane przez `*ngIf` na podstawie roli
- `<ion-router-outlet>` — outlet Angular Routera, tutaj renderowane są strony
- `.chat-fab` — pływający przycisk SVG czatu, widoczny na wszystkich stronach oprócz `/chat`

Centralny rejestr wszystkich tras URL aplikacji. Każda trasa używa `loadComponent` (lazy loading — ładowanie komponentu dopiero gdy potrzebny).

```
{
  path: 'sale/:id/edit',
  canActivate: [authGuard],
  loadComponent: () => import('./features/sale/sale.page').then(m => m.SalePage),
}
```

Wzorzec trasy	Komponent	Guard
<code>/login</code>	LoginPage	brak
<code>/change-password</code>	ChangePasswordPage	brak
<code>/home</code>	HomePage	authGuard
<code>/hermonogram</code>	HermonogramPage	authGuard
<code>/konflikty</code>	KonfliktyPage	authGuard
<code>/sale</code> , <code>/sale/new</code> , <code>/sale/:id/edit</code> , <code>/sale/dictionaries</code>	Sale*	authGuard
<code>/subjects</code> , <code>/subjects/new</code> , <code>/subjects/:id/edit</code> , <code>/subjects/dictionaries</code>	Subject*	authGuard
<code>/teaching-loads</code>	TeachingLoadsPage	authGuard
<code>/admin/users/new</code>	UserCreatePage	authGuard
<code>/admin/unavailability-notes</code>	UnavailabilityNotesListAdmin	authGuard
<code>/chat</code>	ChatPage	authGuard
<code>/audit-logs</code>	AuditLogsPage	authGuard
<code>/profile</code>	ProfilePage	authGuard
<code>' ' / **</code>	redirect → <code>/login</code>	—

3. `core/auth/`

Trzy pliki zarządzające uwierzytelnianiem JWT — warstwa niskopoziomowa operująca bezpośrednio na tokenach.

`auth.service.ts`

`src/app/core/auth/auth.service.ts`

Serwis JWT backendu. Zarządza tokenami i cyklem życia sesji na poziomie HTTP.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private accessToken: string | null = null;
  private readonly currentUserSubject = new BehaviorSubject<AuthUser | null>(null);
  readonly currentUser$ = this.currentUserSubject.asObservable();

  login(payload: LoginRequest): Observable<LoginResponse> { ... }
  refreshAccessToken(): Observable<string> { ... }
  fetchCurrentUser(): Observable<AuthUser> { ... }
  logout(): Observable<void> { ... }
  isAuthenticated(): boolean { return !this.jwtService.isExpired(this.accessToken, 10); }
}
```

Metoda / Pole	Endpoint	Opis
<code>accessToken</code> (prywatne)	—	Token JWT przechowywany wyłącznie w pamięci RAM — nie w localStorage
<code>currentUser\$</code>	—	BehaviorSubject emitujący dane zalogowanego użytkownika
<code>login(payload)</code>	POST <code>/auth/</code>	Logowanie — otrzymuje access token i zapisuje go
<code>refreshAccessToken()</code>	POST <code>/auth/refresh</code>	Odświeżenie tokena — używa cookie HttpOnly
<code>fetchCurrentUser()</code>	GET <code>/auth/me</code>	Pobiera profil zalogowanego użytkownika
<code>logout()</code>	POST <code>/auth/logout</code>	Wylogowanie + <code>clearSession()</code>
<code>isAuthenticated()</code>	—	Sprawdza ważność tokena z buforem 10 sekund
<code>mustChangePassword</code>	—	Flaga wymuszająca zmianę hasła przy pierwszym logowaniu

Dekoder tokenów JWT. Nie używa żadnych zewnętrznych bibliotek — korzysta z wbudowanej funkcji przeglądarki `atob()`.

```
decodePayload(token: string): Record<string, unknown> | null {
  const parts = token.split('.'); // nagłówek.payload.podpis
  const payload = parts[1].replace(/-/g, '+').replace(/_/g, '/');
  return JSON.parse(atob(payload)); // base64 → JSON
}

isExpired(token: string | null, skewSeconds = 0): boolean {
  const expiresAt = this.getExpirationDate(token);
  return expiresAt.getTime() <= Date.now() + skewSeconds * 1000;
}
```

Metoda	Zwraca	Opis
<code>decodePayload(token)</code>	<code>Record<string, unknown></code>	Dekoduje część środkową JWT (Base64URL → JSON)
<code>getExpirationDate(token)</code>	<code>Date null</code>	Czyta pole <code>exp</code> z payloadu i konwertuje Unix timestamp → Date
<code>isExpired(token, skewSec)</code>	<code>boolean</code>	Sprawdza wygaśnięcie z buforem czasowym (domyślnie 0s)
<code>getUserId(token)</code>	<code>number null</code>	Wyciąga <code>user_id</code> z payloadu
<code>getRole(token)</code>	<code>string null</code>	Wyciąga pole <code>role</code> z payloadu

Route Guard. Wykonywany przez router przed wyrenderowaniem każdej chronionej trasy.

```
export const authGuard: CanActivateFn = () => {  
  // 1. Zalogowany + mustChangePassword → /change-password  
  if (authService.isAuthenticated() && authService.mustChangePassword)  
    return router.createUrlTree(['/change-password']);  
  
  // 2. Token ważny → przepuść  
  if (authService.isAuthenticated()) return true;  
  
  // 3. Token wygasł → próba refresh  
  return authService.refreshAccessToken().pipe(  
    map(token => { authService.setAccessToken(token); return true; }),  
    catchError(() => of(router.createUrlTree(['/login'])))  
  );  
};
```

Funkcja (nie klasa)

Nowoczesna forma guardu w Angular 14+ — `CanActivateFn` to zwykła funkcja używająca `inject()` zamiast klasy z konstruktorem.

4. `core/interceptors/`

`auth.interceptor.ts`

`src/app/core/interceptors/auth.interceptor.ts`

Globalny interceptor HTTP. Przechwytuje każde żądanie do `apiBaseUrl`, automatycznie obsługuje wygaśnięcie tokenów.

```
let refreshInFlight$: Observable<AuthRefreshResponse> | null = null;

function runRefresh(http): Observable<...> {
  if (!refreshInFlight$) {
    refreshInFlight$ = http.post('/auth/refresh', {}, { withCredentials: true }).pipe(
      finalize(() => refreshInFlight$ = null),
      shareReplay(1) // <-- deduplicja: wiele 401 = jeden refresh
    );
  }
  return refreshInFlight$;
}
```

Sytuacja	Zachowanie
Żądanie spoza <code>apiBaseUrl</code>	Przepuszczone bez modyfikacji
Żądanie do API	Klonowane z <code>withCredentials: true</code> (wysyła cookie)
Odpowiedź 401 (nie-auth endpoint)	Uruchamia <code>runRefresh()</code> , ponawia żądanie
401 na <code>/auth/refresh</code> lub <code>/auth/logout</code>	Przekierowanie na <code>/login</code> (nie próbuje refresh)
Refresh się nie powiódł	Przekierowanie na <code>/login</code>

`shareReplay(1)` — deduplicja refreshu

Jeśli 5 żądań dostanie 401 jednocześnie, `refreshInFlight$` jest ustawiony tylko przy pierwszym wywołaniu. Kolejne 4 dostają ten sam Observable i czekają na jeden refresh request. Zapobiega wysłaniu 5 requestów do `/auth/refresh`.

5. `core/services/`

13 serwisów dostarczających dane z backendu. Każdy jest `Injectable({ providedIn: 'root' })` — singleton dostępny w całej aplikacji.

auth.service.ts

src/app/core/services/auth.service.ts

Główny serwis sesji frontendowej. To fasada łącząca JWT auth (`core/auth/auth.service.ts`) z logiką UI (rola, display name, avatar, motyw). Istnieją DWA serwisy `auth.service.ts` — ten jest używany przez komponenty UI.

```
export type UserRole = 'admin' | 'lecturer' | 'planner' | 'rapla_editor' | 'lecturer_rapla_editor';
export type AppTheme = 'light' | 'dark';

private _authenticated = new BehaviorSubject<boolean>(false);
private _userRole = new BehaviorSubject<UserRole | null>(null);
private _displayName = new BehaviorSubject<string>('');
readonly isAuthenticated$ = this._authenticated.asObservable();
```

Właściwość / Metoda	Opis
<code>isAuthenticated\$</code>	Observable — menu reaguje automatycznie gdy zmienia się na <code>true/false</code>
<code>role</code>	Getter aktualnej roli użytkownika (<code>'admin'</code> , <code>'lecturer'</code> itp.)
<code>roleLabel</code>	Polska etykieta roli do wyświetlenia w UI
<code>displayName</code>	Imię i nazwisko lub login do wyświetlenia w nagłówku
<code>avatarUrl\$</code>	Observable URL avatara (lub domyślny placeholder)
<code>mustChangePassword</code>	Flaga — przekierowanie na <code>/change-password</code>
<code>login(login, password)</code>	Logowanie — wywołuje backend, ustawia sesję
<code>logout()</code>	Wylogowanie + czyszczenie stanu + nawigacja do <code>/login</code>
<code>restoreBackendSession()</code>	Przy starcie app próbuje odtworzyć sesję przez <code>GET /auth/me</code>
<code>ensureAndUploadPublicKey(userId)</code>	Po zalogowaniu generuje/uploaduje klucz RSA dla E2EE czatu
<code>updateProfileAvatar(dataUrl)</code>	Aktualizuje avatar użytkownika
<code>updateCurrentTheme(theme)</code>	Zmienia motyw i zapisuje w preferencjach

Serwis WebSocket (Socket.IO). Zarządza połączeniem real-time i eksponuje strumienie RxJS dla komponentów czatu.

```
this.socket = io(backendUrl, {
  withCredentials: true,
  auth: token ? { token } : undefined,
  reconnection: true,
  reconnectionAttempts: 5,
  reconnectionDelay: 1000,
});

// Strumienie zdarzeń
public messageReceived$ = new Subject<ChatMessage>();
public userTyping$ = new Subject<TypingIndicator>();
public messageDelivered$ = new Subject<MessageStatus>();
```

Observable	Typ	Zdarzenie WS
isConnected\$	BehaviorSubject<boolean>	Stan połączenia
messageReceived\$	Subject<ChatMessage>	message_received
userTyping\$	Subject<TypingIndicator>	user_typing
messageDelivered\$	Subject<MessageStatus>	message_delivered
messageRead\$	Subject<MessageStatus>	message_read
userOnline\$	Subject<UserPresence>	user_online
userOffline\$	Subject<UserPresence>	user_offline
notification\$	Subject<any>	notification

crypto.service.ts

src/app/core/services/crypto.service.ts

Serwis kryptografii end-to-end. Używa natywnego Web Crypto API przeglądarki — żadnych zewnętrznych bibliotek.

```
// Klucz prywatny w IndexedDB (nigdy nie wychodzi z urządzenia)
await idbSet('rsa_private_jwk', privateKeyJwk);

// Klucz publiczny na backendzie
await http.put(`/users/${userId}/public-key`, { public_key: pemString });

// Szyfrowanie wiadomości
async encryptMessage(plaintext: string, recipientPublicPem: string): Promise<string>

// Odszyfrowanie
async decryptMessage(ciphertext: string): Promise<string>
```

Metoda	Opis
<code>ensureRSAKeyPair()</code>	Sprawdza IndexedDB — jeśli brak klucza, generuje nową parę RSA-OAEP 2048-bit
<code>getPublicPem()</code>	Zwraca klucz publiczny w formacie PEM (<code>-----BEGIN PUBLIC KEY-----</code>)
<code>encryptMessage(text, pubPem)</code>	Szyfruje tekst kluczem publicznym odbiorcy
<code>decryptMessage(cipher)</code>	Deszyfruje tekst własnym kluczem prywatnym z IndexedDB

Pomocnicze funkcje IndexedDB

`idbGet(key)` i `idbSet(key, value)` — minimalne wrappery na IndexedDB API (baza `zpi-crypto`, store `kv`). Klucz prywatny zapisany jako JWK (JSON Web Key).

subjects-api.service.ts

src/app/core/services/subjects-api.service.ts

CRUD przedmiotów dydaktycznych i ich słowników.

```
export interface SubjectDto {
  id: number; name: string; type_id: number | null;
  activity_id: number | null; activity_name: string | null;
  type_display: string | null; room_properties: string | null;
  blocked: boolean; periodic: boolean;
}
```

Metoda	HTTP	Endpoint
<code>getSubjects()</code>	GET	<code>/subjects/list</code>
<code>getSubjectById(id)</code>	GET	<code>/subjects/:id</code>
<code>createSubject(payload)</code>	POST	<code>/subjects/</code>
<code>updateSubject(id, payload)</code>	PATCH	<code>/subjects/:id</code>
<code>deleteSubject(id)</code>	DELETE	<code>/subjects/:id</code>
<code>getActivities()</code>	GET	<code>/activities/list</code>
<code>createActivity(payload)</code>	POST	<code>/activities/</code>
<code>deleteActivity(id)</code>	DELETE	<code>/activities/:id</code>

Normalizacja odpowiedzi

Backend może zwracać `{items: [...]}` lub tablicę bezpośrednio — prywatna metoda `normalizeSubject()` ujednolica oba formaty. Operator `map` w pipe obsługuje oba przypadki.

rooms-api.service.ts

src/app/core/services/rooms-api.service.ts

CRUD sal wykładowych ze słownikami typów, sprzętu i wydziałów.

```
export interface RoomDto {
  id: number; room_number: string; seats_count: number;
  room_type_id: number | null; room_type: string;
  special_equipment: number[]; special_equipment_names: string[];
  activities: number[]; activity_names: string[];
  departments: number[]; department_names: string[];
}
```

Metoda	HTTP	Endpoint
<code>getRooms()</code>	GET	<code>/rooms/list</code>
<code>getRoomById(id)</code>	GET	<code>/rooms/:id</code>
<code>createRoom(payload)</code>	POST	<code>/rooms/</code>
<code>updateRoom(id, payload)</code>	PATCH	<code>/rooms/:id</code>
<code>deleteRoom(id)</code>	DELETE	<code>/rooms/:id</code>
<code>getRoomTypes()</code> , <code>createRoomType()</code> , <code>deleteRoomType(id)</code>	GET/POST/DELETE	<code>/room-types</code>
<code>getSpecialEquipment()</code> , <code>createSpecialEquipment()</code> , <code>deleteSpecialEquipment(id)</code>	GET/POST/DELETE	<code>/special-equipment</code>
<code>getDepartments()</code> , <code>createDepartment()</code> , <code>deleteDepartment(id)</code>	GET/POST/DELETE	<code>/departments</code>

teaching-loads-api.service.ts

src/app/core/services/teaching-loads-api.service.ts

CRUD przydziałów godzin dydaktycznych. Łączy dane z kilku serwisów.

```
export interface TeachingLoadAssignmentDto {
  id: number; teacher_id: number; teacher_first_name: string | null;
  teacher_last_name: string | null; subject_id: number;
  subject_name: string | null; activity_id: number;
  activity_name: string | null; semester_id: number;
  semester_name: string | null; hours: number;
}
```

Importuje z innych serwisów

Importuje `AuditApiService`, `SubjectsApiService`, `UsersAdminApiService` — łączy dane z wielu endpointów do jednego widoku tabeli.

dezyderata.service.ts

src/app/core/services/dezyderata.service.ts

Zarządzanie dezyderatami (preferencjami dostępności wykładowców) i semestrami.

```
export interface Dezyderata {
  id: number; user_id: number; semestr_id: number;
  day_id: number; from_hour: number; to_hour: number;
  is_available: boolean; day_name?: string;
}

getSemestry(): Observable<SemestrListResponse>
getCurrentSemestr(): Observable<Semestr>
getMyDezyderaty(semestrId?): Observable<DezyderataListResponse>
createDezyderata(payload: DezyderataCreate): Observable<...>
deleteDezyderataEntry(id): Observable<void>
```

unavailability-notes-api.service.ts

src/app/core/services/unavailability-notes-api.service.ts

API notatek o niedostępności wykładowców.

```
export type NoteType = 'request' | 'forced';
export type NoteStatus = 'pending' | 'accepted' | 'rejected' | 'acknowledged';

createNote(payload): Observable<UnavailabilityNoteDto> // POST /unavailability-notes/
getMyNotes(): Observable<UnavailabilityNoteDto[]> // GET /unavailability-notes/m
getAllNotes(): Observable<UnavailabilityNoteListDto[]> // GET /unavailability-notes/a
getPendingNotes(): Observable<...> // GET /unavailability-notes/a
updateNoteStatus(id, status): Observable<...> // PATCH /unavailability-notes/
```

notifications-api.service.ts

src/app/core/services/notifications-api.service.ts

Powiadomienia użytkownika — pobieranie i zarządzanie statusem przeczytania.

```
getNotifications(limit=50, offset=0, unreadOnly=false): Observable<NotificationListDto>
getUnreadNotifications(): Observable<NotificationListDto>
markAsRead(id): Observable<NotificationDto>
markAllAsRead(): Observable<{message: string}>
getUnreadCount(): Observable<UnreadCountDto>
```

audit-api.service.ts

src/app/core/services/audit-api.service.ts

Pobieranie logów audytowych i potwierdzanie ich obejrzenia.

```
export interface AuditLogDto {
  id: number; entity_name: string; entity_id: number;
  action: string; old_values: any; new_values: any;
  modified_by: number; modified_by_name: string;
  timestamp: string; isNew?: boolean; // <-- pole frontendowe
}

getLogs(): Observable<AuditLogResponseDto> // GET /audit-logs
acknowledgeChanges(): Observable<{message: string}> // POST /audit-logs/acknowledge
```

Pole **isNew**

To pole nie pochodzi z backendu — jest obliczane frontendowo przez porównanie **timestamp** logu z **last_changes_viewed_at** z odpowiedzi. Umożliwia wyróżnienie nowych wpisów w UI.

rapla-api.service.ts

src/app/core/services/rapla-api.service.ts

Integracja z systemem Rapla (system planowania uczelni).

```
getReservations(): Observable<RaplaReservationDto[]> // GET /rapla/reservations

importFile(file: File): Observable<ImportSummary> {
  const fd = new FormData();
  fd.append('file', file, file.name);
  return this.http.post('/rapla/file/import', fd, ...);
}
```

Interfejs **RaplaReservationDto** zawiera: id, uuid, name, color, reservation_type, start_date, start_time, end_date, end_time, repeating_type, allocate, room_names, teacher_names, semester_names, group_names.

users-admin-api.service.ts

src/app/core/services/users-admin-api.service.ts

API zarządzania użytkownikami — dostępne tylko dla roli admin.

```
export interface AdminUserRow {
  user_id: number; album_number: string; first_name: string;
  last_name: string; login: string; email: string;
  titles: string[]; roles: string[]; departments: string[];
  must_change_password: boolean;
}

getUsers(): Observable<AdminUserRow[]>
createUser(payload: AdminCreateUserPayload): Observable<AdminCreatedUserResponse>
updateUser(id, payload: AdminUpdateUserPayload): Observable<AdminUserRow>
deleteUser(id): Observable<void>
resetUserPassword(id): Observable<{login, one_time_password}>
getRoles(), getDepartments(), getTitles() // słowniki
```

projects-api.service.ts

src/app/core/services/projects-api.service.ts

Prosty serwis pobierający listę projektów.

```
export interface ProjectDto { id: number; name: string; }
getProjects(): Observable<ProjectDto[]> // GET /api/projects
```

6. `core/validators/form-validators.ts`

form-validators.ts

src/app/core/validators/form-validators.ts

Biblioteka własnych walidatorów Angular Reactive Forms. Każdy walidator zwraca `ValidatorFn` (funkcję).

```
const NAME_PATTERN = /^[a-zA-ZąęłńóśźżĄĆĘŁŃÓŚŻŻ][a-zA-ZąęłńóśźżĄĆĘŁŃÓŚŻŻ\s\-\-]{1,99}$/;
const EMAIL_PATTERN = /^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$/;
```

Eksportowana funkcja	Reguła walidacji	Błąd gdy
<code>nameValidator()</code>	Polskie znaki, myślnik, spacja, 2–100 znaków	<code>{ nameInvalid: true }</code>
<code>strictEmailValidator()</code>	Format email z dokładnym regex	<code>{ email: true }</code>
<code>passwordStrengthValidator()</code>	≥8 znaków, wielka litera, mała litera, cyfra, znak specjalny	<code>{ missingUppercase: true }</code> itp.
<code>passwordMatchValidator(key1, key2)</code>	Dwa pola muszą być równe (poziom FormGroup)	<code>{ passwordMismatch: true }</code>
<code>passwordNotContainsNameValidator(first, last)</code>	Hasło nie może zawierać imienia ani nazwiska	<code>{ containsName: true }</code>
<code>calcPasswordStrength(value)</code>	Funkcja pomocnicza (nie walidator) — zwraca 0–4	—

7. `pages/` — Strony wejściowe

`login.page.ts``src/app/pages/login/login.page.ts`

Strona logowania. Dostępna bez autoryzacji (`/login`). Template-driven form.

```
login = ''; password = ''; error = ''; isLoading = false;

ngOnInit() {
  if (this.auth.isLoggedIn) this.router.navigateByUrl('/home'); // już zalogowany → skip
}

submit() {
  this.auth.login(this.login, this.password).subscribe({
    next: (res) => {
      if (res.mustChangePassword) this.router.navigateByUrl('/change-password');
      else this.router.navigateByUrl('/home');
    },
    error: () => this.error = 'Nieprawidłowe dane logowania'
  });
}
```

- Pole hasła z przełącznikiem widoczności (`showPassword`) + ikony `eyeOutline` / `eyeOffOutline`)
- Spinner `isLoading` blokuje przycisk podczas oczekiwania na API
- Komunikat błędu `error` wyświetlany w UI

Wymuszona zmiana hasła. Reactive Form z wieloma walidatorami.

```
readonly form = this.fb.group({
  newPassword: ['', [Validators.required, passwordStrengthValidator()]],
  confirmPassword: ['', [Validators.required]],
},
{ validators: passwordMatchValidator('newPassword', 'confirmPassword') }
);

// Dynamicznie dodany walidator po załadowaniu imienia/nazwiska
this.form.controls.newPassword.addValidators(
  passwordNotContainsNameValidator(this.auth.firstName, this.auth.lastName)
);
```

Getter	Opis
<code>passwordStrength</code>	Liczba 0–4 obliczona przez <code>calcPasswordStrength()</code>
<code>passwordStrengthLabel</code>	<code>'Słabe'</code> / <code>'Średnie'</code> / <code>'Mocne'</code> / <code>'Bardzo mocne'</code>
<code>passwordStrengthClass</code>	Klasa CSS do kolorowania paska siły hasła

8. `features/home/` — Dezyderaty

`home.page.ts`

`src/app/features/home/pages/home.page.ts`

Najobszerniejszy komponent aplikacji. Obsługuje pełną funkcjonalność dezyderatów z widokiem tygodniowym kalendarza.

```
type AvailabilityMode = 'available' | 'unavailable';

interface LecturerStatusItem {
  userId: number; lecturerDisplayName: string;
  departments: string[]; isApproved: boolean;
  submission: LecturerSubmission | null;
  hasActiveUnavailability: boolean;
}

interface TutorialStep {
  target: 'mode-selector' | 'calendar-cell' | 'confirm-button' | 'edit-button' | 'clear-t';
  title: string; description: string;
}
```

Kluczowy element	Opis
<code>AvailabilityMode</code>	Tryb oznaczania: dostępny / niedostępny
<code>LecturerStatusItem</code>	Stan dezyderaty jednego wykładowcy (widok admina/planisty)
<code>LecturerSubmission</code>	Zapisane sloty dostępności: dni × godziny
<code>TutorialStep</code>	Krok interaktywnego samouczka pierwszego użycia
<code>HistoryWeekItem</code>	Element historii złożonych dezyderatów (archiwum)
<code>WeekDay</code>	Dzień tygodnia w siatce kalendarza (nazwa, ISO date, isToday)
<code>DezyderataService</code>	CRUD dezyderatów i semestrów
<code>UnavailabilityNotesApiService</code>	Notatki niedostępności powiązane z dezyderatami
<code>UsersAdminApiService</code>	Lista wykładowców (widok admina)
<code>ModalController</code>	Otwiera <code>UnavailabilityNoteModalComponent</code>
<code>raplaFileUrl</code>	URL do pobrania pliku Rapla (<code>\${apiBaseUrl}/rapla/file</code>)
<code>@HostListener('document:keydown')</code>	Obsługa skrótów klawiaturowych w siatce

9. `features/hermonogram/` — Harmonogram

`hermonogram.page.ts`

`src/app/features/hermonogram/pages/hermonogram.page.ts`

Tygodniowy widok harmonogramu z importem Rapla.

```
interface RaplaRenderedItem {
  reservation: RaplaReservationDto;
  topOffsetMinutes: number; // pozycja Y w siatce
  durationMinutes: number; // wysokość bloku
  startMin: number; endMin: number;
  colIndex: number; colCount: number; // obsługa kolizji (nakładanie)
}

hours24 = Array.from({ length: 15 }, (_, i) => i + 7); // [7, 8, ..., 21]
weekDays: WeekDay[] = []; // 5 lub 7 dni tygodnia
```

Interfejs / Pole	Opis
<code>RaplaRenderedItem</code>	Rezerwacja z obliczoną pozycją CSS na siatce czasowej
<code>WeekDay</code>	Dzień tygodnia z nazwą, ISO datą i flagą <code>isToday</code>
<code>ImportedPlanEntry</code>	Zaimportowany plan z localStorage (legacy)
<code>miniCalendarDays</code>	Dni mini-kalendarza do nawigacji
<code>RaplaApiService</code>	Import pliku i pobieranie rezerwacji z backendu
<code>selectedDate</code>	Aktualnie wybrana data — determinuje wyświetlany tydzień

10. `features/konflikty/` — Konflikty

`konflikty.page.ts`

`src/app/features/konflikty/pages/konflikty.page.ts`

Strona wyświetlająca konflikty harmonogramu. Minimalna logika — głównie UI z menu profilowym.

```
constructor(public readonly auth: AuthService, private readonly router: Router) {}

get userRoleLabel() { return this.auth.roleLabel; }
get userDisplayName() { return this.auth.displayName; }

logout() { this.auth.logout(); }
openSettings() { this.router.navigateByUrl('/profile'); }
```

11. `features/sale/` — Sale wykładowe

`sale-list.page.ts`

`src/app/features/sale/sale-list.page.ts`

Lista sal z CRUD. Ochrona ról wewnątrz `ngOnInit`.

```
ngOnInit(): void {
  if (this.auth.role !== 'admin') {
    this.router.navigateByUrl('/home'); return; // guard wewnętrzny
  }
  this.loadRooms();
}

deleteRoom(roomId: number): void {
  const accepted = window.confirm('Czy na pewno chcesz usunąć te sale?');
  if (!accepted) return;
  this.roomsApi.deleteRoom(roomId).pipe(finally(() => this.isDeleting = false)).subscribe()
}
```

Pola stanu: `isLoading`, `isDeleting`, `errorMessage`, `rooms: RoomDto[]`.

`sale.page.ts`

`src/app/features/sale/sale.page.ts`

Formularz tworzenia i edycji sali. Tryb edycji wykrywany przez parametr `:id` w URL.

```
ngOnInit(): void {
  const idParam = this.route.snapshot.paramMap.get('id');
  this.roomId = idParam ? Number(idParam) : null;
  this.isEditMode = this.roomId !== null;
  this.loadInitialData(); // forkJoin: typy, sprzęt, wydziały, aktywności
  if (this.isEditMode) this.loadRoom(this.roomId!); // wypełnij formularz
}

loadInitialData(): void {
  forkJoin([
    this.roomsApi.getRoomTypes(),
    this.roomsApi.getActivities(),
    this.roomsApi.getSpecialEquipment(),
    this.roomsApi.getDepartments(),
  ]).subscribe([types, activities, equipment, departments] => { ... })
}
```

forkJoin

RxJS operator — odpala wiele żądań HTTP równolegle i czeka aż **wszystkie** się zakończą, zwraca tablicę wyników. Używany do ładowania słowników (zamiast 4 sekwencyjnych requestów).

sale-dictionaries.page.ts

src/app/features/sale/sale-dictionaries.page.ts

Zarządzanie słownikami sal: typy, sprzęt specjalny, wydziały, aktywności. CRUD dla każdego słownika z potwierdzeniem przez `AlertController`.

sale-dictionaries.ts

src/app/features/sale/sale-dictionaries.ts

Plik re-eksportu: `export { SaleDictionariesPage } from './sale-dictionaries.page'`. Istnieje dla uproszczenia ścieżki importu w `app.routes.ts`.

12. `features/subjects/` — Przedmioty

subject-list.page.ts

src/app/features/subjects/subject-list.page.ts

Lista przedmiotów. Używa `ionViewWillEnter` (Ionic lifecycle hook) zamiast `ngOnInit` do odświeżania danych — wykonuje się za każdym wejściem na stronę (nie tylko przy pierwszym).

```
ionViewWillEnter(): void {  
  this.loadSubjects(); // odświeża listę przy każdym powrocie na stronę  
}
```

subject.page.ts

src/app/features/subjects/subject.page.ts

Formularz tworzenia/edycji przedmiotu. Template-driven form (`NgForm`).

```
name = ''; activityId: number | null = null;
typeDisplay = ''; roomProperties = '';
blocked = false; periodic = true;

onSubmit(form: NgForm): void {
  if (!form.valid) { this.errorMessage = 'Uzupełnij wszystkie wymagane pola'; return; }
  const payload: SubjectPayload = {
    name: this.name.trim(),
    activity_id: Number(this.activityId),
    type_display: this.typeDisplay || null,
    room_properties: this.roomProperties || null,
    blocked: this.blocked,
    periodic: this.periodic,
  };
  const req$ = this.isEditMode
    ? this.subjectsApi.updateSubject(this.subjectId!, payload)
    : this.subjectsApi.createSubject(payload);
  req$.subscribe(...)
}
```

unavailability-note-modal.component.ts

src/app/features/subjects/components/unavailability-note-modal/

Modal Ionic do dodawania notatek niedostępności z poziomu strony przedmiotu. Reactive Form.

```
private initializeForm(): void {
  this.form = this.fb.group({
    noteType: ['request', Validators.required], // 'request' | 'forced'
    startDate: ['', Validators.required],
    endDate: [''],
    description: ['', Validators.maxLength(1000)],
  });
}

dismiss() { this.modalController.dismiss(); } // zamknij bez zapisu

submit() {
  const payload: CreateUnavailabilityNoteRequest = { ...this.form.value };
  this.unavailabilityService.createNote(payload).subscribe(
    result => this.modalController.dismiss(result) // zamknij z wynikiem
  );
}
```

13. `features/teaching-loads/` — Przydziały godzin

`teaching-loads.page.ts`

`src/app/features/teaching-loads/teaching-loads.page.ts`

Zaawansowana tabela z inline edycją i historią zmian.

```
type TeachingLoadFilterState = {
  teacher_id?: number | null; subject_id?: number | null;
  activity_id?: number | null; semester_id?: number | null;
  hours_min?: number | null; hours_max?: number | null;
};

// Stan edycji inline
editingRowId: number | null = null;
draftRow: TeachingLoadAssignmentDto | null = null; // kopia edytowanego wiersza
originalRow: TeachingLoadAssignmentDto | null = null; // oryginał do porównania

// Stan tworzenia nowego wiersza
isCreating = false;
newRowDraft: TeachingLoadAssignmentDto | null = null;

// Historia zmian per wiersz
historyOpenRowId: number | null = null;
historyByAssignment: Record<number, AuditLogDto[]> = {};
```

Kluczowa funkcja	Opis
Inline edit	Kliknięcie wiersza ustawia <code>editingRowId</code> + klonuje wiersz do <code>draftRow</code> ; zapis PATCH tylko jeśli <code>draftRow !== originalRow</code>
Filtry wielopoziomowe	<code>TeachingLoadFilterState</code> — filtrowanie client-side po 6 kryteriach
Historia per wpis	Klik → GET <code>/audit-logs?entity=TeachingLoadAssignment&id=X</code> → rozwinięcie pod wierszem
<code>forkJoin</code> w <code>ngOnInit</code>	Ładuje równolegle: przydziały, nauczycieli, przedmioty, aktywności, semestry

14. `features/admin/`

`user-create.page.ts``src/app/features/admin/pages/user-create.page.ts`

Panel zarządzania użytkownikami — tworzenie, edycja, reset hasła. Dwa formularze reaktywne.

```
// Formularz tworzenia
readonly form = this.fb.group({
  firstName: ['', [nameValidator()]],
  lastName: ['', [nameValidator()]],
  email: ['', [strictEmailValidator()]],
  oneTimePassword: ['', [Validators.required, Validators.minLength(8)]],
  roleIds: this.fb.control<number[]>([], { validators: [Validators.required] }),
  departmentIds: this.fb.control<number[]>([], { validators: [Validators.required] }),
  titleId: this.fb.control<number | null>(null),
});

// Tabela użytkowników z paginacją
readonly pageSizeOptions = [5, 10, 25, 50];
searchQuery = ''; pageSize = 10; currentPage = 1;

// Stan modalów
isEditModalOpen = false;
isResetPasswordModalOpen = false;
selectedUser: AdminUserRow | null = null;
```

Shared Credentials

Po stworzeniu/resecie hasła backend zwraca jednorazowe dane logowania (`login` + `one_time_password`) które admin przekazuje użytkownikowi. Wyświetlane w interfejsie jako `createdCredentials` / `resetCredentials` .

**unavailability-notes-list-
admin.component.ts**src/app/features/admin/pages/unavailability-notes-
list-admin/

Lista notatek niedostępności dla admina. Filtrowanie po statusie przez segmenty.

```
type FilterType = 'all' | 'pending' | 'accepted' | 'rejected' | 'acknowledged';
filterType: FilterType = 'all';

// Obiekt blokowania – zapobiega kliknięciu przycisku dla tej samej notatki dwa razy
isProcessing: { [noteId: number]: boolean } = {};

// Zmiana statusu
updateStatus(noteId: number, status: NoteStatus): void {
  this.isProcessing[noteId] = true;
  this.unavailabilityService.updateNoteStatus(noteId, { status })
    .pipe(finalize(() => this.isProcessing[noteId] = false))
    .subscribe(() => this.ionViewWillEnter()); // odśwież listę
}
```

Implementuje **ViewWillEnter** (Ionic) — odświeża dane za każdym wejściem na stronę.

15. `features/chat/` — Czat E2EE

7 komponentów + 1 serwis + 1 plik modeli. Najbardziej rozbudowany moduł aplikacji.

`chat.models.ts`

`src/app/features/chat/models/chat.models.ts`

Centralna definicja wszystkich typów używanych w module czatu.

Interfejs / Typ	Opis
<code>ChatUser</code>	Użytkownik: <code>user_id</code> , <code>first_name</code> , <code>last_name</code> , <code>public_key?</code> , <code>avatarUrl?</code>
<code>ChatRoom</code>	Pokój grupowy: <code>id</code> , <code>name</code> , <code>members: number[]</code> , <code>createdAt</code>
<code>ChatMessage</code>	Wiadomość: <code>id</code> , <code>senderId</code> , <code>content</code> , <code>isOwn</code> , <code>isRead</code>
<code>MessageApiResponse</code>	Surowa odpowiedź API: <code>ciphertext?</code> , <code>iv?</code> , <code>wrapped_key?</code> , <code>encrypted_aes_key?</code> — pola E2EE
<code>ConversationListResponse</code>	Konwersacja: <code>type: 'direct' 'group'</code> , <code>member_ids</code>
<code>MessageStatusApiResponse</code>	Status wiadomości: <code>sent</code> , <code>delivered</code> , <code>read</code> z timestampami
<code>ActiveTab</code>	Union type: <code>'messages' 'contacts' 'create-room'</code>
<code>ChatView</code>	Union type: <code>'contacts' 'conversation' 'room' 'create-room'</code>
<code>OpenChat</code>	Otwarta karta czatu w trybie page: <code>id</code> , <code>user?</code> , <code>room?</code> , <code>messages[]</code>

chat-api.service.ts

src/app/features/chat/services/chat-api.service.ts

REST API czatu — wszystkie endpointy `/api/chat/`.

Metoda	HTTP	Endpoint
<code>searchUsers(query)</code>	GET	<code>/api/chat/search-users?q=...</code>
<code>getAvailableUsers(limit)</code>	GET	<code>/api/chat/search-users?limit=100</code>
<code>getConversations()</code>	GET	<code>/api/chat/conversations</code>
<code>startConversation(userId)</code>	POST	<code>/api/chat/start-conversation/:userId</code>
<code>createGroup(name, userIds)</code>	POST	<code>/api/chat/create-group</code>
<code>getConversationMessages(convId, beforeId?, limit?)</code>	GET	<code>/api/chat/:id/messages</code>
<code>sendMessage(convId, content)</code>	POST	<code>/api/chat/:id/send-message</code>
<code>markMessageAsRead(messageId)</code>	POST	<code>/api/chat/messages/:id/read</code>
<code>getPublicKey(userId)</code>	GET	<code>/users/:id/public-key</code>
<code>getUserAvatar(userId)</code>	GET	<code>/users/:id/avatar</code>

Główny komponent czatu. Zarządza całą logiką konwersacji, szyfrowania i WebSocket.

```
// Signals (Angular 16+ reactive state)
isOpen = signal(false);
activeTab = signal<ActiveTab>('messages');
view = signal<ChatView>('contacts');
selectedUser = signal<ChatUser | null>(null);
selectedRoom = signal<ChatRoom | null>(null);
openChats = signal<OpenChat[]>([]); // tryb page – wiele otwartych czatów

// Auto-scroll
@ViewChild('messagesContainer') messagesContainer!: ElementRef;
private shouldScrollToBottom = false;
ngAfterViewChecked() { if (this.shouldScrollToBottom) scrollToBottom(); }

// Wyszukiwanie z debounce
private searchSubject = new Subject<string>();
// pipe(debounceTime(300), distinctUntilChanged(), switchMap(q => search(q)))
```

Kluczowy mechanizm	Opis
Angular Signals	Nowy reaktywny prymityw Angular 16+ — <code>signal()</code> zamiast <code>BehaviorSubject</code> dla lokalnego stanu
<code>debounceTime(300)</code>	Wyszukiwanie odpala API dopiero 300ms po ostatnim znaku — redukuje zbędne requesty
<code>distinctUntilChanged()</code>	Pomija duplicate zapytania (identyczna wartość)
<code>switchMap</code>	Anuluje poprzedni request jeśli użytkownik pisze dalej
<code>ViewEncapsulation.None</code>	Style CSS komponentu działają globalnie (potrzebne dla dynamicznie dodawanego contentu)
<code>@Input() pageMode</code>	Tryb pełnoekranowy (strona /chat) vs. panel boczny — różne zachowanie

Komponenty chat/components/

src/app/features/chat/components/

Pięć małych komponentów prezentacyjnych ("**dumb components**") — nie mają własnej logiki, otrzymują dane przez `@Input()` i emitują zdarzenia przez `@Output()`.

Komponent	@Input()	@Output()
chat-panel-header	view, selectedUser, selectedRoom, getInitials	back, close
chat-panel-tabs	view, activeTab	tabChange
chat-contacts-view	searchQuery, searchResults, isSearching, directContacts, getInitials, getLastMessage, isLastMessageUnread	searchQueryChange, search, openConversation
chat-rooms-view	rooms, getLastRoomMessage	showCreateRoom, openRoom
chat-create-room-view	newRoomName, newRoomMembers, availableUsers, isMemberSelected, getInitials	newRoomNameChange, toggleRoomMember, createRoom

Smart vs. Dumb components

`chat.component.ts` jest "smart" — ma całą logikę i dane. Komponenty w `components/` są "dumb" — tylko renderują i emitują zdarzenia. Wzorzec poprawia testowalność i czytelność kodu.

chat.page.ts

src/app/features/chat/pages/chat.page.ts

Wrapper strony — dodaje Ionic header z menu i osadza `<app-chat [pageMode]="true">`.

Minimalna logika — tylko menu profilowe i nawigacja.

16. `features/audit/` — Historia zmian

`audit-logs.page.ts`

`src/app/features/audit/pages/audit-logs.page.ts`

Wyświetla audit logi z grupowaniem i translacją nazw na polski.

```
interface LogBatch {
  key: string;           // "user_id-date"
  userName: string;      // "Jan Kowalski"
  timeLabel: string;     // "12:34"
  logs: AuditLogDto[];
  hasNew: boolean;       // czy są nieprzeczytane
}

interface LogGroup {
  date: string;          // "2026-05-12"
  batches: LogBatch[];
}

// Mapa tłumaczeń technicznych nazw modeli → polskie
readonly entityMap: Record<string, string> = {
  'Subject': 'Przedmiot',
  'Room': 'Sala',
  'TeachingLoadAssignment': 'Przydział godzin',
  'UnavailabilityNote': 'Notatka o niedostępności',
  // ...
};

// Mapa tłumaczeń nazw pól
readonly translateMap: Record<string, string> = {
  'user_id': 'Użytkownik (ID)',
  'teacher_id': 'Wykładowca (ID)',
  // ...
};
```

Implementuje `ViewWillEnter` — wywołuje `acknowledgeChanges()` po wejściu na stronę aby zaznaczyć na backendzie że zmiany zostały obejrzone.

17. `features/profile/` — Profil

`profile.page.ts``src/app/features/profile/pages/profile.page.ts`

Strona ustawień profilu użytkownika.

```
triggerAvatarUpload(fileInput: HTMLInputElement) {
  fileInput.click(); // programatyczne otwarcie dialogu wyboru pliku
}

async onAvatarFileSelected(event: Event) {
  const file = input.files?.[0];
  const dataUrl = await this.readFileAsDataURL(file); // FileReader API
  this.auth.updateProfileAvatar(dataUrl).subscribe(); // upload na backend
}

ionViewWillEnter() {
  this.selectedTheme = this.auth.getCurrentTheme(); // synchronizacja motywu
}

onThemeChange(isDark: boolean) {
  this.selectedTheme = isDark ? 'dark' : 'light';
  this.auth.updateCurrentTheme(this.selectedTheme);
}
```

Pola formularza zmiany hasła: `currentPassword`, `newPassword`, `confirmPassword`.